

Allure + Schemathesis Integration Guide

1. Audience & Scope

- **Audience:** QA engineers wiring **Patrol E2E** with **Allure + Schemathesis** API contract tests.
 - **Scope:** Configure, run, and tune Allure, Schemathesis, and their integration (incl. token export) in this repo or a new project.
-

2. Folder Layout & Key Files

- `lib/testing/token_export.dart` — app helper that writes a **JWT access token** into the app sandbox when built with `--dart-define=EXPORT_TOKEN=true`.
- `integration_test/allure_schemathesis_reporting/`
 - `run_patrol_allure.sh` — Patrol + Allure runner (Android & iOS) with optional **Schemathesis import & deeplink**.
 - `run_schemathesis.sh` — wrapper for `schemathesis run` (**JUnit + HAR** only).
 - `run_schemathesis_allure.sh` — **Schemathesis → Allure** pipeline (separate Allure report).
 - `st_aggregate_to_allure.py` — converts Schemathesis **JUnit + HAR** into **Allure results** (1 test per endpoint + API health).
 - `patrol_env.sh` — optional env file auto-sourced by the runner.
 - `sct_auth/get_schemathesis_token.sh` — device-independent **OAuth2** token fetch.

- `sct_auth/ios_pull_token.sh` — pulls `.schemathesis_token` from **iOS Simulator**.
 - `schemathesis_severity.json` — per-endpoint **risk mapping** (critical/normal/low).
 -  When porting to a new project, keep structure, but update: **PKG/BUNDLE_ID**, **ST_URL/BASE_URL**, **ST_SCHEMA** (e.g., `openapi-docs.yaml`), and `schemathesis_severity.json` to match your risk model.
-

3. Prerequisites

- **CLI tools:** Patrol CLI; Allure CLI (`allure --version`); Python 3.
 - **Mobile tooling:** Android device/emulator (`adb`); iOS **Xcode + CLT** for `xcrun simctl` (if using Simulator).
 - **Flutter app change:** include `lib/testing/token_export.dart`; wire auth flow to call `TokenExport.save()` after login (see `docs/Schemathesis_Token_Export.md`).
-

4. Token Export & Authentication

4.1 App-side export

- Enable with `--dart-define=EXPORT_TOKEN=true` (or `EXPORT_TOKEN=1` via script).
- After successful login, the app writes `.schemathesis_token` in its sandbox:
 - **Android:** `/data/data/<PKG>/app_flutter/.schemathesis_token`
 - **iOS Simulator:** `<AppContainer>/Documents/.schemathesis_token`

4.2 Host-side token (two modes)

- **export_token** (from device): Patrol test + app export → scripts pull to host as `./schemathesis_token`.
 - **direct_api** (device-independent): `sct_auth/get_schemathesis_token.sh` calls your auth API and prints a token.
 - Both modes are supported by `run_patrol_allure.sh` and `run_schemathesis_allure.sh`.
-



5. Allure + Patrol (UI Tests Only)

Main script:

```
integration_test/allure_schemathesis_reporting/run_patrol_allure.sh
```

5.1 🗝️ Important env vars

- **Core:** `TARGET`, `PLATFORM` (`android/ios/auto`), `TAGS`, `EXCLUDE_TAGS`, `SERVE_REPORT` (1 to run `allure serve`).
- **Environment panel:** `APP_VERSION`, `FLAVOR`, `APP_ENV`, `BASE_URL`, `GIT_SHA`, `DEVICE_NAME`, `API_LEVEL`.

5.2 ▶️ Running (no Schemathesis)

```
./integration_test/allure_schemathesis_reporting/run_patrol_allure.sh \
integration_test/logIn_test.dart
```

```
./integration_test/allure_schemathesis_reporting/run_patrol_allure.sh
```

- **Results:** Allure results → `build/reports/allure-results/<timestamp>*/`; HTML → `build/allure-report/index.html`. (Archive the HTML in CI.)

5.3 🤖 Android configuration (project setup)

- **Instrumentation runner** (`android/app/build.gradle`):

```
testInstrumentationRunner =  
    "com.thinslices.solarisdemo.AllurePatrolJUnitRunner"
```

- **Allure deps** (`androidTest`):

```
androidTestImplementation "io.qameta.allure:allure-kotlin-model:2.4.0"  
androidTestImplementation "io.qameta.allure:allure-kotlin-commons:2.4.0"  
androidTestImplementation "io.qameta.allure:allure-kotlin-junit4:2.4.0"  
androidTestImplementation "io.qameta.allure:allure-kotlin-android:2.4.0"
```

- **UiAutomator**:

```
androidx.test.uiautomator:uiautomator:2.2.0
```

- **Orchestrator** (optional):

```
androidTestUtil "androidx.test:orchestrator:1.5.1"
```

- **Enable TestStorage** (`androidTest/resources/allure.properties`):

```
allure.results.useTestStorage=true
```

- **Helper code (key files)**: `AllureEnrichmentRule.kt`, `FilteredLogcatRule.kt`,
`AllureStepHandler.kt` for labels/links/params and attachments.

5.4 🍏 iOS configuration (allure-xcresult)

- Convert Xcode `.xcresult` → Allure with **allure-xcresult** (binary `AllureXCResult` or via repo).
- Runner looks under `./build` for the latest `.xcresult`, converts to Allure results, and populates simulator/OS info into the Environment panel when possible.

- Example variables supported by the script:

```
ALLURE_XCRESULT_BIN=/usr/local/bin/AllureXCResult
ALLURE_XCRESULT_REPO=/path/to/allure-xcresult    # uses: swift run -c
release AllureXCResult
```

6. Schemathesis as a Standalone Tool

Two options: **raw CLI** (`run_schemathesis.sh`) or the **full Allure pipeline** (`run_schemathesis_allure.sh`) — recommended.

6.1 `run_schemathesis_allure.sh`

Core env:

- `ST_URL` (API base, required), `ST_SCHEMA` (OpenAPI path, default `openapi-docs.yaml`), `SCEMATHEISIS_BIN` (optional), `AUTO_VENV=1` (create venv under `.../myenv`).

Generation controls:

- `ST_PHASES` (e.g., `examples, coverage, stateful`), `ST_MODE` (`all/positive`), `ST_WORKERS` (concurrency), `ST_WEIGHT` (`auto` or `explicit`).

Auth modes:

- `GET_TOKEN_MODE=export_token` (default; pull from device) or `direct_api` (call OAuth2 script).
- `PKG`, `BUNDLE_ID` for pulling; `AUTH_SCRIPT` for direct API (default `sct_auth/get_schemathesis_token.sh`).

Outputs:

- Raw run → `build/schemathesis-allure/run/` (**JUnit + HAR + `output.txt`**)
- Allure results → `build/schemathesis-allure/allure-results/`
- HTML → `build/allure-report-schemathesis/`

- Health summary →
`build/schemathesis-allure/allure-results/schemathesis_summary.json`

6.2 Examples

- Token from device:

```
AUTO_VENV=1 \  
GET_TOKEN_MODE=export_token \  
ST_URL="https://your-api.example.com" \  
./integration_test/allure_schemathesis_reporting/run_schemathesis_allure.sh
```

- Token via auth API:

```
AUTO_VENV=1 \  
GET_TOKEN_MODE=direct_api \  
ST_URL="https://your-api.example.com" \  
ST_AUTH_URL="https://your-auth.example.com/oauth2/token" \  
ST_AUTH_CLIENT_ID="your-client-id" \  
ST_AUTH_CLIENT_SECRET="..." \  
ST_AUTH_USERNAME="..." \  
ST_AUTH_PASSWORD="..." \  
./integration_test/allure_schemathesis_reporting/run_schemathesis_allure.sh
```

Tune depth vs speed:

- More depth/fuzz → `ST_PHASES="examples,coverage,fuzzing,stateful"`,
`ST_MODE="all"`, higher `ST_WORKERS`.
- Fast sanity → `ST_MODE="positive"`, `ST_PHASES="examples,coverage"`.

7. Combined Flow: Patrol + Schemathesis + Deeplink

1. **Login & export token via Patrol** (no Schemathesis yet):

```
PLATFORM=android \  
EXPORT_TOKEN=1 \  
RUN_ST=0 \  
IMPORT_ST_INTO_PATROL=0 \  
SERVE_REPORT=0 \  
./integration_test/allure_schemathesis_reporting/run_patrol_allure.sh \  
integration_test/logIn_test.dart
```

2. **Run Schemathesis + Allure** using that token:

```
AUTO_VENV=1 \  
GET_TOKEN_MODE=export_token \  
ST_URL="https://your-api.example.com" \  
./integration_test/allure_schemathesis_reporting/run_schemathesis_allure.sh
```

...or use `GET_TOKEN_MODE=direct_api` with the relevant auth env vars (same script).

3) **Run Patrol again to refresh E2E report and attach the Schemathesis deeplink:**

```
PLATFORM=android \  
RUN_ST=0 \  
IMPORT_ST_INTO_PATROL=1 \  
SERVE_REPORT=0 \  
./integration_test/allure_schemathesis_reporting/run_patrol_allure.sh \  
integration_test/logIn_test.dart
```

After step 3 you'll have:

- Patrol Allure report: [build/allure-report/index.html](#)
- Schemathesis Allure report: [build/allure-report/schemathesis/index.html](#)
- Deeplink from the Patrol report to the Schemathesis report (via Allure's **Executor** panel)



Using Each Piece Independently (not code)

- **Allure only (UI tests):** run `run_patrol_allure.sh` with `RUN_ST=0`; ignore Schemathesis variables.
 - **Schemathesis only:** run `run_schemathesis_allure.sh` directly; choose `GET_TOKEN_MODE`.
 - **No-auth APIs:** use `export_token` mode without exporting a token, or add a `none` mode; Schemathesis will run without `Authorization`.
-



.gitignore (Recommended — not code)

- Local venv: `integration_test/allure_schemathesis_reporting/myenv/`
- Schemathesis outputs: `integration_test/schemathesis-report/`,
`build/schemathesis-allure/`
- Allure reports: `build/allure-report/`, `build/allure-report-schemathesis/`
- Sensitive token file: `.schemathesis_token`